

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using `fork()`. 3. Execute the command in the child process using an appropriate `exec()` system call. 4. Allow the parent process to wait for the child using `wait()`. 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (`uname`, `lscpu`, `lsblk`, `ps`, `top`), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main() {
    char cmd[10];
    pid_t pid;

    printf("Enter the Command\n");
    scanf("%s", cmd);

    pid = fork();

    if (pid == 0) {
        execlp(cmd, cmd, NULL);
        printf("exec() system call failed\n");
        exit(1);
    } else {
        wait(NULL);
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
    }

    return 0;
}
```

```
venkatesh@LAPTOP-LL9RQ023:~$ nano week1.c
venkatesh@LAPTOP-LL9RQ023:~$ cat week1.c

#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main() {
    char cmd[10];
    pid_t pid;

    printf("Enter the Command\n");
    scanf("%s", cmd);

    pid = fork();

    if (pid == 0) {
        execlp(cmd, cmd, NULL);
        printf("exec() system call failed\n");
        exit(1);
    } else {
        wait(NULL);
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
    }

    return 0;
}

venkatesh@LAPTOP-LL9RQ023:~$ gcc week1.c
venkatesh@LAPTOP-LL9RQ023:~$ ./a.out.c
-bash: ./a.out.c: No such file or directory
venkatesh@LAPTOP-LL9RQ023:~$ ./a.out
Enter the Command
12
exec() system call failed
Parent PID: 595
Child PID: 596
venkatesh@LAPTOP-LL9RQ023:~$ |
```